BÖLÜM 7 / MODULE 7

Asenkron Boru Hatları & CompletableFuture

Asynchronous Pipelines & CompletableFuture

Geleneksel Future engellemelerinden (blocking get) kurtuluş, CompletableFuture ile reaktif veri akışları, thenCompose/thenCombine bileşimleri, allOf/anyOf orkestrasyonu, hata kurtarma zincirleri ve zaman aşımı yönetimi.

Escaping traditional blocking Future traps, CompletableFuture reactive processing pipelines, thenCompose/thenCombine functional chains, allOf/anyOf orchestration, resilient error recovery, and timeout handling.

37

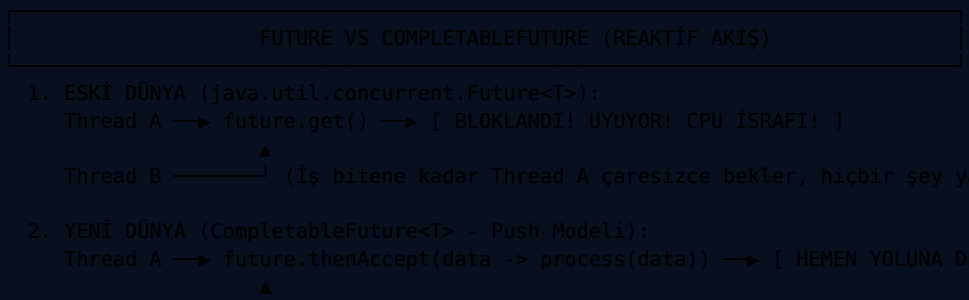
BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Future<T> vs CompletableFuture<T>: Polling Kâbusundan Reaktif İtmeye Geçiş

Future<T> vs CompletableFuture<T>: Escaping the Polling Nightmare

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski Future, pizza siparişi verip kapının deliğine gözünü dayayarak 45 dakika boyunca hiçbir yere kımıldamadan kuryeyi beklemeye benzer (`future.get()` işi bitene kadar thread'i kilitler). CompletableFuture ise kapıya 'Pizza gelince zili çal, pizzayı fırına atıp kutuyu çöpe at' yazılı bir akıllı talimat yapıştırmaktır. Kurye geldiğinde talimatlar otomatik yürütülür, sen o sırada salonda televizyon izlemeye devam edersin!

🇬🇧 ENGLISH (ELI5)

The legacy `Future\<T>` is like ordering food and standing motionlessly with your eye glued to the door peephole for 45 minutes (`future.get()` blocks the thread dead). `CompletableFuture\<T>` is leaving an intelligent sticky note on the door: 'When the courier arrives, put pizza on the table, tip $5, and text me'. You continue watching television. When the data is ready, callbacks push reactions forward without blocking any CPU thread!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Java 5'te gelen `Future\<T>` arayüzünün 3 devasa zafiyeti vardı: 1) Bloklayan Çekme Modeli: Sonucu almak için `future.get()` çağrılır ve çağıran thread uyutulur (Context Switch maliyeti). 2) Zincirleme Yapılamaz: Bir Future'ın çıktısını diğer bir Future'ın girdisi yapacak deklaratif yapı yoktur. 3) Manuel Tamamlama İmkânsızdır: Dışarıdan bir etkinlik (web-socket, JMS mesajı) geldiğinde Future'ı el ile sonuçlandıramazsınız. Java 8 ile gelen `CompletableFuture\<T>`, `CompletionStage` ve `Future` arayüzlerini birleştirir. ForkJoinPool.commonPool() üzerinde veya özel thread havuzlarında asenkron görevleri non-blocking (olay güdümlü) zincirler halinde yürütür.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Shortcomings of legacy `Future\<T>`: 1) Blocking Polling Trap: Callers are trapped in synchronous blocking via `future.get()`, squandering expensive kernel threads. 2) Absence of Declarative Chaining: Cannot natively express 'execute task B when task A completes'. 3) Immutable Completion: Callers cannot externally resolve or fail a Future instance. `CompletableFuture\<T>` implements both `Future\<T>` and `CompletionStage\<T>`. It adopts the Promise pattern: tasks push transformations down a directed acyclic graph (DAG) executed on `ForkJoinPool.commonPool()` or custom `Executor` contexts without idling caller threads.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Non-blocking CompletableFuture ile Sipariş İşleme
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.Executors;

public class NonBlockingOrderPipeline {
    record Order(String id, double amount, String status) {}

    public static void main(String[] args) {
        try (var customPool = Executors.newFixedThreadPool(4)) {
            // Asenkron veri çekme başlat (Non-blocking!)
            CompletableFuture<Order> orderFuture = CompletableFuture.supplyAsync(() -> {
                System.out.println("Sipariş çekiliyor: " + Thread.currentThread());
                return new Order("ORD-9821", 250.0, "PENDING");
            }, customPool);

            // Reaktif zincir: Veri geldiğinde otomatik işlet (Bloklama yok!)
            orderFuture
                .thenApply(order -> new Order(order.id(), order.amount() * 1.20, "TAX_INCLUDED"))
                .thenAccept(processed -> {
                    System.out.println("İşlem Tamamlandı: " + processed + " Thread: " +
Thread.currentThread());
                });

            System.out.println("Ana thread asla bloklanmadı, diğer işlere devam ediyor...");
            Thread.sleep(100); // Demo amaçlı arka plan işinin bitmesini bekle
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Argümansız supplyAsync(Supplier) varsayılan olarak ForkJoinPool.commonPool() kullanır! Ağ/veritabanı gibi bloklayan (I/O-bound) işleri commonPool'a yüklerseniz havuzdaki tüm CPU çekirdeklerini kilitler ve sistemdeki paralel stream'leri felç edersiniz. I/O işleri için daima özel bir Executor (veya Java 21+ Virtual Thread Executor) geçin!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Executing `supplyAsync()` without an explicit executor defaults to `ForkJoinPool.commonPool()`. Flooding the common pool with blocking I/O (database, REST calls) exhausts CPU-worker threads, starving parallel streams and application background tasks. Always supply a dedicated I/O thread pool or a Virtual Thread executor!

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Future.get() çağrısı thread'i kilitler. CompletableFuture ise reaktif itme (push) modelidir; iş bittiğinde sonraki aşama otomatik tetiklenir.

🎯 Key Takeaways for the Interview

• `Future.get()` blocks threads synchronously. `CompletableFuture` employs non-blocking event-driven completion stages via push semantics.

38

BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Dönüşüm Boru Hatları: thenApply vs thenAccept vs thenRun & Thread Havuzu Geçişleri

Transformation Pipelines: thenApply vs thenAccept vs thenRun & Async Variants

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│                DÖNÜŞÜM OPERATÖRLERİ KARŞILAŞTIRMASI          │
└──────────────────────────────────────────────────────────────┘

Operatör        | Girdi Alır mı?   | Çıktı Üretir mi? | Fonksiyonel Arayüz
----------------|------------------|------------------|-------------------
thenApply(f)    | EVET (T)         | EVET (U)         | Function<T, U>
thenAccept(c)   | EVET (T)         | HAYIR (void)     | Consumer<T>
thenRun(r)      | HAYIR (void)     | HAYIR (void)     | Runnable

*Async Farkı:
stage.thenApply(fn)      ──► Önceki görevi bitiren thread AYNI thread'de devam eder.
stage.thenApplyAsync(fn) ──► Görevi Executor kuyruğuna atar, BAŞKA bir thread çalıştırır!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

1) thenApply: Aşçıya eti verirsin, pişirip biftek olarak geri verir (Girdi var, dönüşüm var). 2) thenAccept: Garsona bifteği verirsin, afiyetle yer veya müşteriye sunar, sana bir şey geri vermez (Girdi var, çıktı yok). 3) thenRun: İki işlem de bitince aşçı mutfaktaki gongu çalar; ne etle ilgilenir ne tabakla, sadece 'İş bitti!' der (Girdi yok, çıktı yok). Sonuna 'Async' eklersen bu işi başka bir personele devredersin!

🇬🇧 ENGLISH (ELI5)

1) `thenApply`: Handing raw flour to a baker and receiving bread back (consumes `T`, returns `U`). 2) `thenAccept`: Handing bread to a customer who consumes it without returning a value (consumes `T`, returns `void`). 3) `thenRun`: The kitchen clock ringing at closing time; independent of data (consumes `void`, returns `void`). Appending `*Async` (e.g., `thenApplyAsync`) dispatches execution to an Executor queue rather than running inline on the completing thread.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Senkron ve Asenkron varyantlar arasındaki kritik ayrım: `stage.thenApply()` çağrıldığında, eğer önceki aşama zaten tamamlanmışsa fonksiyon çağıran thread üzerinde çalışır. Önceki aşama henüz bitmemişse, o aşamayı tamamlayan iş parçacığı (worker thread) üzerinde inline olarak çalıştırılır; böylece gereksiz context-switch önlenir. `stage.thenApplyAsync(fn, executor)` ise işi kesinlikle belirtilen executor'ın iş kuyruğuna teslim eder. Uzun süren CPU hesaplamaları veya bağımsız thread context'i (örneğin MDC/SecurityContext) gerektiren adımlar için `*Async` varyantları tercih edilmelidir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Inline Execution vs Async Dispatch: In `stage.thenApply(fn)`, if the preceding stage has already completed, `fn` executes synchronously on the calling thread. If pending, it executes inline on whichever worker thread completes the upstream stage, eliminating queue dispatch overhead. Conversely, `thenApplyAsync(fn, pool)` guarantees task submission into the designated executor's run-queue. Use `*Async` methods when the transform logic is CPU-intensive or requires clean thread isolation.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Pipeline Aşamaları ve Özel Executor Geçişi
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.Executors;

public class PipelineTransformationDemo {
    record UserProfile(long id, String rawName, String email) {}
    record EnrichedProfile(long id, String formattedName, boolean isVerified) {}

    void main() {
        try (var cpuPool = Executors.newFixedThreadPool(2);
             var ioPool = Executors.newVirtualThreadPerTaskExecutor()) {

            CompletableFuture.supplyAsync(() -> {
                // I/O işlemi: Veritabanından profili çek
                System.out.println("1. DB Fetch: " + Thread.currentThread());
                return new UserProfile(42L, "  ada lovelace  ", "ada@lovelace.org");
            }, ioPool)
            // Dönüşüm (Function): String temizleme & zenginleştirme (CPU Pool'a devret)
            .thenApplyAsync(user -> {
                System.out.println("2. CPU Transform: " + Thread.currentThread());
                return new EnrichedProfile(user.id(), user.rawName().trim().toUpperCase(), true);
            }, cpuPool)
            // Tüketim (Consumer): Bildirim gönder veya logla
            .thenAccept(enriched -> {
                System.out.println("3. Notification Sent: " + enriched);
            })
            // Tetikleme (Runnable): Metrik sayacını artır
            .thenRun(() -> {
                System.out.println("4. Pipeline tamamlandı, metrik güncellendi.");
            }).join(); // Test için ana thread'i beklet
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

thenApply içine asla bloklayan I/O yazmayın! Eğer bir önceki işlem hızlıca bitmişse, thenApply çağıran ana thread (örneğin Tomcat HTTP worker thread) üzerinde çalışıp ana isteği kilitler. Bloklayan her adım için thenApplyAsync ve dedicated I/O pool kullanın.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Never execute blocking I/O inside `thenApply`! If the upstream stage finished prior to registration, `thenApply` executes directly on the caller thread (e.g., HTTP request thread), causing catastrophic thread stalling. Reserve blocking operations for `thenApplyAsync`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• thenApply veri dönüştürür (T->U), thenAccept veriyi tüketir (T->void), thenRun sadece sinyal verir (void->void). *Async varyantları işi havuza devreder.

🎯 Key Takeaways for the Interview

• `thenApply` transforms data (T->U), `thenAccept` consumes (T->void), `thenRun` signals completion (void->void). `*Async` submits to an executor.

39

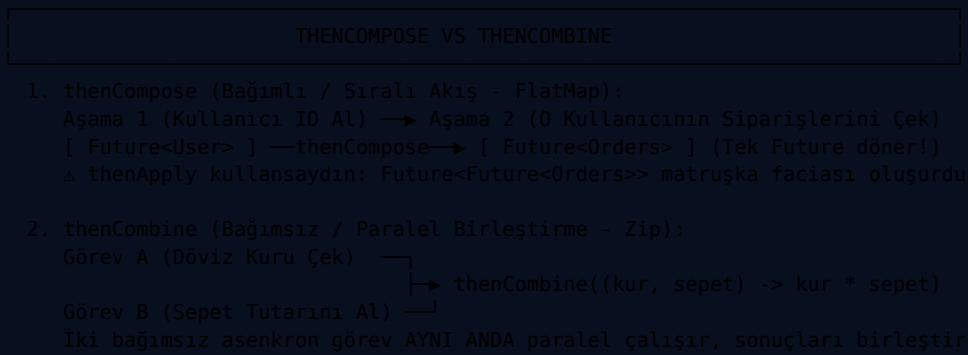
BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Bileşim ve Birleştirme: thenCompose (FlatMap) vs thenCombine (Zip) Farkı

Composition & Merging: thenCompose (FlatMap) vs thenCombine (Zip)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

thenCompose: Bir dedektif gibidir. Önce anahtarın nerede olduğunu bulur (1. aşama), sonra o anahtarla kasayı açar (2. aşama). Biri bitmeden diğeri başlayamaz; iki adımı tek bir temiz sonuca bağlar. thenCombine ise düğün pastası hazırlamak gibidir: Biri pastayı fırınlarken diğeri süsleme kremasını çırpar (iki bağımsız iş aynı anda çalışır). İkisi de bitince pastayla kremayı birleştirirler!

🇬🇧 ENGLISH (ELI5)

`thenCompose` is sequential dependency (monadic flatMap): Step 1 retrieves the encryption key; Step 2 uses that key to decrypt the vault payload. Step 2 cannot begin until Step 1 yields its output. `thenCombine` is parallel aggregation (applicative zip): Worker A calculates flight pricing while Worker B fetches hotel availability concurrently. When both arrive, their results are combined into a travel package.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Monadik Yapı ve Tip İmzaları: 1) `thenCompose(Function\<T, CompletionStage\<U>>)`: Fonksiyonel programlamadaki `flatMap` operatörünün birebir karşılığıdır. Eğer `thenApply` kullansaydınız `CompletableFuture\<CompletableFuture\<U>>` şeklinde içiçe geçmiş (nested) bir tür elde ederdiniz. `thenCompose` bu katmanı düzleştirir (flattening). 2) `thenCombine(CompletionStage\<U>, BiFunction\<T, U, V>)`: İki bağımsız `CompletableFuture`'ı paralel olarak koşturur. Her ikisi de başarıyla tamamlandığında `BiFunction`'ı tetikleyerek iki sonucu tek bir `V` değerinde sentezler. Biri çökerse birleşik future da çöker.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Monadic Anatomy: 1) `thenCompose`: Represents functional `flatMap`. If you map an asynchronous function `T -> CompletableFuture\<U>` via `thenApply`, you produce a nested `CompletableFuture\<CompletableFuture\<U>>`. `thenCompose` flattens the hierarchy, binding stages sequentially. 2) `thenCombine`: Functional `zipWith`. Executes both stages asynchronously in parallel. Upon dual completion, it supplies both outputs `(T, U)` to a `BiFunction\<T, U, V>`. If either stage aborts exceptionally, the aggregated stage terminates with `ExecutionException`.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: thenCompose (Sıralı) ve thenCombine (Paralel) Kullanımı
import java.util.concurrent.CompletableFuture;

public class CompositionAndCombineDemo {
    record User(long id, String creditTier) {}
    record CreditLimit(long userId, double limitUsd) {}
    record Cart(long userId, double totalUsd) {}
    record CheckoutDecision(boolean approved, String reason) {}

    static CompletableFuture<User> fetchUser(long id) {
        return CompletableFuture.supplyAsync(() -> new User(id, "PREMIUM"));
    }

    static CompletableFuture<CreditLimit> fetchLimit(User user) {
        return CompletableFuture.supplyAsync(() ->
            new CreditLimit(user.id(), user.creditTier().equals("PREMIUM") ? 10000.0 : 1000.0)
        );
    }

    static CompletableFuture<Cart> fetchCart(long userId) {
        return CompletableFuture.supplyAsync(() -> new Cart(userId, 3200.0));
    }

    void main() {
        long userId = 101L;

        // 1. thenCompose: User -> CreditLimit (Bağımlı Sıralı Akış)
        CompletableFuture<CreditLimit> limitFuture = fetchUser(userId)
            .thenCompose(CompositionAndCombineDemo::fetchLimit);

        // 2. Sepeti bağımsız paralel çek
        CompletableFuture<Cart> cartFuture = fetchCart(userId);

        // 3. thenCombine: Limit ile Sepeti birleştirip karar ver (Paralel Zip!)
        CompletableFuture<CheckoutDecision> decisionFuture = limitFuture.thenCombine(cartFuture,
            (limit, cart) -> {
                boolean ok = cart.totalUsd() <= limit.limitUsd();
                return new CheckoutDecision(ok, ok ? "Onaylandı" : "Kredi limiti yetersiz!");
            });

        CheckoutDecision decision = decisionFuture.join();
        System.out.println("Ödeme Kararı: " + decision);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

thenCombine kullanırken iki görevin birbirini beklemediğinden emin olun. Eğer Görev A'nın girdisi Görev B'ye lazımsa thenCombine ÇALIŞMAZ; o zaman mecburen thenCompose kullanmalısınız. Yanlış operatör seçimi deadlock veya derleme hatalarına yol açar.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

`thenCombine` mandates that both futures are strictly independent. If stage B requires an argument generated by stage A, using `thenCombine` is invalid; you must use `thenCompose`. Misusing them creates cascading compile-time type mismatches.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• thenCompose bağımlı asenkron adımları birbirine zincirler (flatMap). thenCombine bağımsız iki asenkron görevi paralel çalıştırıp birleştirir (zip).

🎯 Key Takeaways for the Interview

• `thenCompose` sequences dependent async stages (flatMap). `thenCombine` executes two independent stages in parallel and fuses their outputs (zip).

40

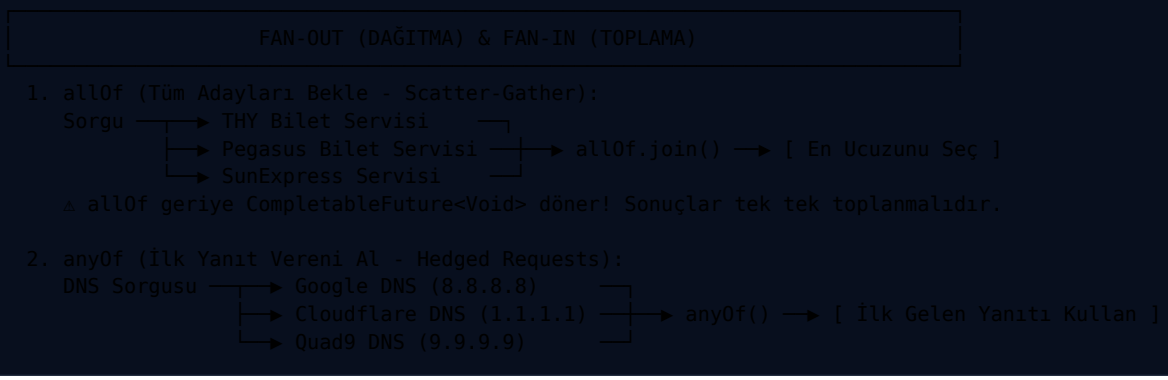
BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Fan-Out & Fan-In Orkestrasyonu: CompletableFuture.allOf vs anyOf ile Dağıtık Koordinasyon

Fan-Out & Fan-In Coordination: CompletableFuture.allOf vs anyOf

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

allOf: Bir otobüs şoförünün tüm yolcuların (uçak bileti arayan 5 farklı sitenin) binmesini beklemesi gibidir. Herkes binmeden otobüs kalkmaz; sonra yolcuların biletlerini sayarsın. anyOf ise bir taksi durağında sıraya girip ilk gelen boş taksiye binmektir. 3 taksiyi birden çağırırsın, kapıya ilk yanaşan taksiye biner gidersin; diğerleri boşa çıkar!

🇬🇧 ENGLISH (ELI5)

`CompletableFuture.allOf` is scatter-gather: A travel aggregator querying 5 airline APIs concurrently. The master request waits until every single airline responds before filtering the lowest price. `CompletableFuture.anyOf` is hedged requests: Querying 3 redundant DNS mirrors (Google, Cloudflare, Quad9) simultaneously and immediately returning whichever responds first, discarding trailing packets.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HotSpot allOf ve anyOf İç Mantığı: 1) `allOf(CompletableFuture\<?>... cfs)`: Dahili olarak bir `BiRelay` ağacı kurar. Her bir future tamamlandığında bir sayaç eksiltilir. Sayaç 0'a ulaştığında bileşik future tetiklenir. DÖNÜŞ TİPİ `CompletableFuture\<Void>`'dir! Sonuçları almak için orijinal future dizisi stream edilip `CompletableFuture::join` ile toplanır (bu noktada tümü bittiği için join ASLA bloklamaz). 2) `anyOf(CompletableFuture\<?>... cfs)`: İlk tamamlanan (veya ilk istisna fırlatan) future bileşik nesneyi çözer. DÖNÜŞ TİPİ `CompletableFuture\<Object>`'tir ve tip dönüşümü (cast) gerektirir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Implementation Details: 1) `allOf`: Constructs a binary tree of completion relays. Each completing leaf decrements an atomic countdown latch. Returns `CompletableFuture\<Void>` by design to accommodate heterogeneous return types. To extract payloads, stream over the input list mapping via `CompletableFuture::join`—which is guaranteed non-blocking because all tasks are already resolved. 2) `anyOf`: Binds an identical completion handler to all upstream stages; the first node to transition invokes `completeRelay()`. Returns `CompletableFuture\<Object>`, requiring explicit downcasting.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: allOf ile Scatter-Gather Fiyat Toplama
import java.util.List;
import java.util.concurrent.CompletableFuture;

public class ScatterGatherAggregator {
    record PriceQuote(String provider, double price) {}

    static CompletableFuture<PriceQuote> fetchQuote(String provider, double price, long delayMs) {
        return CompletableFuture.supplyAsync(() -> {
            try { Thread.sleep(delayMs); } catch (InterruptedException _) {}
            return new PriceQuote(provider, price);
        });
    }

    void main() {
        var futures = List.of(
            fetchQuote("Airline-A", 180.0, 50),
            fetchQuote("Airline-B", 150.0, 80),
            fetchQuote("Airline-C", 210.0, 30)
        );

        // allOf ile tüm servislerin bitmesini bekle
        CompletableFuture<Void> allDone = CompletableFuture.allOf(
            futures.toArray(new CompletableFuture[0])
        );

        // Non-blocking toplama (join garantili olarak anında döner!)
        CompletableFuture<List<PriceQuote>> quotesFuture = allDone.thenApply(_ ->
            futures.stream().map(CompletableFuture::join).toList()
        );

        List<PriceQuote> quotes = quotesFuture.join();
        PriceQuote bestQuote = quotes.stream()
            .min((q1, q2) -> Double.compare(q1.price(), q2.price()))
            .orElseThrow();

        System.out.println("En İyi Teklif: " + bestQuote);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

allOf içinde tek bir future patlarsa (exception), allOf da anında patlar! Ancak diğer çalışan future'lar arka planda çalışmaya ve kaynak tüketmeye devam eder. Dağıtık sistemlerde allOf kullanırken her alt task'ın içine 'exceptionally' koyarak default fallback üretmesini sağlayın.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

If any child future fails inside `allOf`, the composite future fails immediately. However, surviving peer futures continue running in the background, squandering resources. Always attach `.exceptionally()` or fallback handlers to individual stages before passing them into `allOf`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• allOf tüm görevlerin bitmesini bekler (scatter-gather, Void döner). anyOf ilk biteni alır (hedged request, Object döner).

🎯 Key Takeaways for the Interview

• `allOf` waits for all stages to conclude (scatter-gather, returns `Void`). `anyOf` yields the first finished stage (hedged calls, returns `Object`).

41

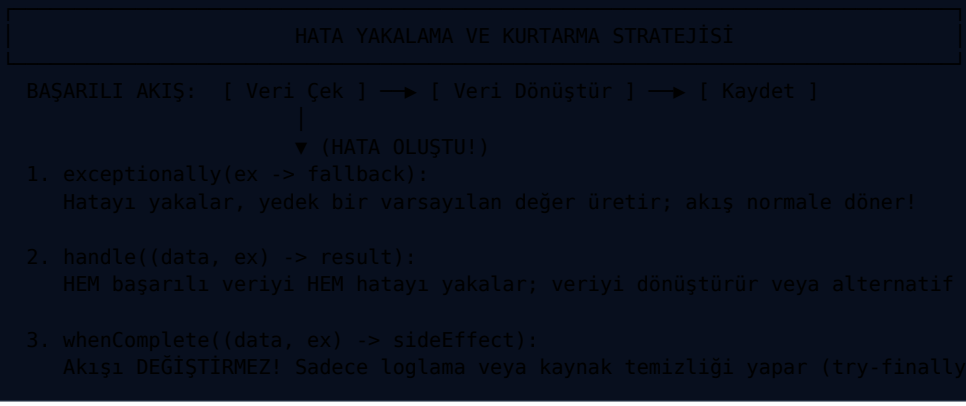
BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Hata Kurtarma Zincirleri: exceptionally vs handle vs whenComplete ile Dayanıklılık

Resilience & Exception Handling: exceptionally vs handle vs whenComplete

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kargo teslimatı düşünün: 1) exceptionally: Müşteri evde yoksa kargoyu kapıcıya bırakmaktır (hata olursa yedek plan devreye girer, paket teslim edilmiş sayılır). 2) handle: Kuryenin kapıyı çaldıktan sonra 'Paketi teslim ettim' veya 'Evde yoktu, şubeye yönlendirdim' diye merkez sisteme rapor hazırlamasıdır (hem başarıyı hem hatayı tek bir yeni sonuca çevirir). 3) whenComplete: Paket teslim edilse de kaybolsa da kuryenin araba günlüğüne 'Bu adrese uğrandı' diye imza atmasıdır (akışı değiştirmez, sadece log tutar).

🇬🇧 ENGLISH (ELI5)

1) `exceptionally`: Like a fallback generator. If the primary power grid cuts out, the backup diesel generator fires up seamlessly, supplying replacement power downstream. 2) `handle`: An inspection checkpoint. It receives both `(result, throwable)`. Whether the product is pristine or shattered, it packs it into an inspected report object for downstream consumers. 3) `whenComplete`: Pure side-effect auditing (like `finally`). It logs whether the transaction succeeded or threw an error, but re-emits the exact upstream state downstream without alteration.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Metot İmzaları ve İstisna Kapsülleme (CompletionException): CompletableFuture içindeki kontrol edilmeyen hatalar HotSpot tarafından otomatik olarak `CompletionException` ile sarılır. 1) `exceptionally(Function\<Throwable, T>)`: Yalnızca istisna durumunda tetiklenir. İstisnayı yutar ve geriye geçerli bir `T` değeri döndürerek pipeline'ı kurtarır. 2) `handle(BiFunction\<T, Throwable, U>)`: İstisna olsun veya olmasın her durumda tetiklenir. Eğer `ex != null` ise hata işlenir, aksi halde `res` işlenir. Yeni bir `U` tipi döndürür. 3) `whenComplete(BiConsumer\<T, Throwable>)`: Gözlemcidir (side-effect). Geriye değer döndürmez (`void`). Orijinal stage'in sonucunu veya istisnasını aynen sonraki aşamalara geçirir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Exception Handling Mechanics: Unchecked exceptions in asynchronous pipelines are wrapped in `CompletionException`. 1) `exceptionally`: Invoked exclusively on stage failure. Catches the `Throwable`, recovers with an alternative value `T`, and transitions the downstream stage to normal completion. 2) `handle`: Invoked unconditionally on dual outcomes (`T result, Throwable ex`). Transforms either state into `U`. 3) `whenComplete`: Passive terminal/audit stage taking `BiConsumer\<T, Throwable>`. It executes side-effects (metrics, auditing) without intercepting or altering the original value/exception payload.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: CompletableFuture Dayanıklı Hata Kurtarma
import java.util.concurrent.CompletableFuture;

public class ResilientPipelineDemo {
    record Price(String symbol, double amount, boolean isFallback) {}

    static CompletableFuture<Double> fetchLivePriceFromExchange(String symbol) {
        return CompletableFuture.supplyAsync(() -> {
            if ("BTC".equals(symbol)) {
                throw new IllegalStateException("Kripto Borsası Çöktü (503 Service Unavailable)!");
            }
            return 65000.0;
        });
    }

    void main() {
        String symbol = "BTC";

        CompletableFuture<Price> priceFuture = fetchLivePriceFromExchange(symbol)
            // 1. exceptionally ile hatayı yakala ve önbellekten fallback dön!
            .exceptionally(ex -> {
                System.err.println("⚠ Hata Yakalandı: " + ex.getMessage() + ". Önbelleğe
dönülüyor...");
                return 62000.0; // Cached fallback price
            })
            // 2. handle ile veriyi modelle
            .handle((val, ex) -> {
                boolean isFallback = (val == 62000.0);
                return new Price(symbol, val, isFallback);
            })
            // 3. whenComplete ile metrik logla (akışı bozmaz!)
            .whenComplete((price, ex) -> {
                System.out.println("📊 Audit Log: Fiyat başarıyla belirlendi -> " + price);
            });

        Price finalPrice = priceFuture.join();
        System.out.println("Sonuç: " + finalPrice);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

exceptionallyAsync kullanmazsanız, hata anında fallback üretme kodu hatanın patladığı worker thread üzerinde çalışır. Fallback kodu veritabanına veya diske gidiyorsa thread'i kilitler; bu tür I/O kurtarmalarında exceptionallyCompose veya exceptionallyAsync tercih edilmelidir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Unless `exceptionallyAsync` is used, the fallback logic executes synchronously on whichever worker thread encountered the upstream exception. If your fallback invokes blocking I/O (e.g. disk or cache reads), use `exceptionallyComposeAsync` to avoid carrier thread exhaustion.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• exceptionally hatayı varsayılan değerle kurtarır. handle hem hatayı hem başarıyı dönüştürür. whenComplete ise yan etkisiz loglama yapar.

🎯 Key Takeaways for the Interview

• `exceptionally` recovers from failure with a fallback. `handle` translates dual states into a new value. `whenComplete` performs side-effects without altering state.

42

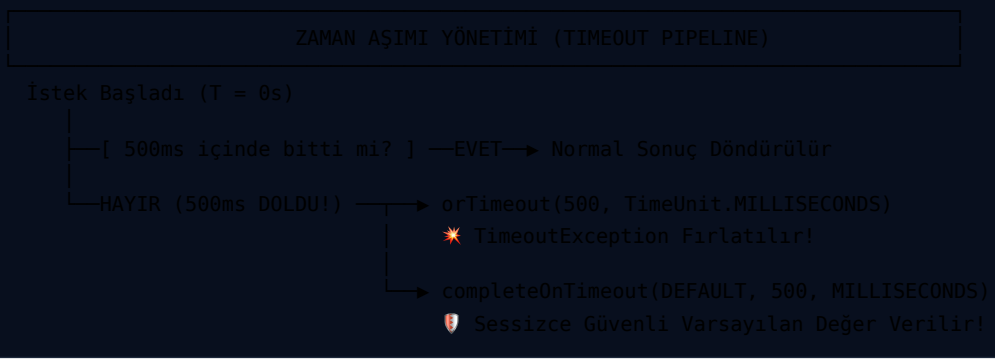
BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Zaman Aşımı ve Varsayılanlar: orTimeout() vs completeOnTimeout() ile Gecikme Yönetimi

Timeouts & Default Fallbacks: orTimeout() and completeOnTimeout()

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoranda sipariş verdiniz ve garsona dediniz ki: 1) orTimeout: '15 dakikada yemeğim gelmezse masayı terk eder, bağırarak çıkarım!' (Süre dolunca ortalığı ayağa kaldıran bir TimeoutException patlar). 2) completeOnTimeout: '15 dakikada biftek pişmezse bana hazır olan soğuk sandviçi getirin, karnımı doyurayım' demektir (Süre dolunca kimse üzülmez, önceden belirlenen yedek sandviç servis edilir).

🇬🇧 ENGLISH (ELI5)

`orTimeout()` is hard SLA enforcement: 'If data does not arrive in 200ms, abort the mission and throw a `TimeoutException` immediately!' `completeOnTimeout()` is graceful degradation: 'If the recommendation AI does not respond within 50ms, return a default list of trending items so the homepage never loads blank'.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Java 9+ ve Java 25 Zaman Aşımı Mimarisi: Java 8'de CompletableFuture için zaman aşımı desteği yoktu; harici bir `ScheduledExecutorService` ile geleceğe cancel/complete basmak gerekiyordu. Modern Java iki güçlü yöntem sunar: 1) `orTimeout(long timeout, TimeUnit unit)`: Süre aşıldığında future'ı `java.util.concurrent.TimeoutException` ile anormal şekilde tamamlar (`completeExceptionally`). 2) `completeOnTimeout(T value, long timeout, TimeUnit unit)`: Belirtilen süre dolduğunda eğer future henüz tamamlanmamışsa, verilen `value` ile normal olarak tamamlanır (`complete(value)`). Bu mekanizma arka planda HotSpot'un paylaşımlı `Delayer` zamanlayıcı thread'ini kullanır; her talep için yeni thread açmaz.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Java 9+ Timeout Architecture: Prior to Java 9, developers had to manually schedule cancellation tasks via `ScheduledExecutorService`. Modern CompletableFuture incorporates: 1) `orTimeout()`: If incomplete when duration elapses, it forces exceptional completion with `TimeoutException`. 2) `completeOnTimeout()`: If incomplete upon timer expiration, it transitions the future to normal completion using the designated fallback object. Both methods register non-blocking delays using an internal daemon `CompletableFuture.Delayer` wheel timer, avoiding per-timeout thread allocations.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: completeOnTimeout ile MAANG Servislerinde Graceful Degradation
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.TimeUnit;

public class TimeoutManagementDemo {
    record Recommendation(String productId, String category) {}

    static CompletableFuture<Recommendation> fetchAiRecommendations() {
        return CompletableFuture.supplyAsync(() -> {
            try {
                // Yapay zeka servisinin 800ms geciktiğini simüle et
                Thread.sleep(800);
            } catch (InterruptedException _) {}
            return new Recommendation("PROD-AI-99", "Personalized");
        });
    }

    void main() {
        Recommendation defaultFallback = new Recommendation("PROD-POPULAR-01", "Trending Now");

        // 200 ms SLA: 200 ms içinde gelmezse defaultFallback değerini bas!
        CompletableFuture<Recommendation> recoFuture = fetchAiRecommendations()
            .completeOnTimeout(defaultFallback, 200, TimeUnit.MILLISECONDS)
            .thenApply(reco -> {
                System.out.println("Kullanıcıya Gösterilen Ürün: " + reco);
                return reco;
            });

        Recommendation result = recoFuture.join();
        System.out.println("Ekrana Basılan Nihai Sonuç: " + result);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

completeOnTimeout çağrısı arka planda geciken asenkron işlemi İPTAL ETMEZ! Arka plandaki işçi thread geciken HTTP çağrısını yapmaya devam edebilir. Gerçekten soket veya thread iptali gerekiyorsa future.cancel(true) veya HTTP client zaman aşımları yapılandırılmalıdır.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

`completeOnTimeout` does NOT cancel the underlying upstream task! The worker thread running the database or HTTP call continues until completion in the background. To abort remote sockets, configure client-level timeouts alongside CompletableFuture.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- orTimeout süre dolduğunda TimeoutException fırlatır. completeOnTimeout ise sessizce yedek bir varsayılan değerle aşamayı tamamlar.

🎯 Key Takeaways for the Interview

- `orTimeout` aborts exceptionally with `TimeoutException`. `completeOnTimeout` silently injects a safe fallback upon SLA breach.